

■ 3.3.4 Algebraic Operations on Polynomials

For many kinds of practical calculations, the only operations you will need to perform on polynomials are essentially the structural ones discussed in the preceding sections.

If you do more advanced algebra with polynomials, however, you will have to use the algebraic operations discussed in this section.

You should realize that most of the operations discussed in this section work only on ordinary polynomials, with integer exponents, and rational number coefficients for each term.

<code>PolynomialQuotient[p, q, x]</code>	find the result of dividing the polynomials p and q in x , dropping any remainder term
<code>PolynomialRemainder[p, q, x]</code>	find the remainder from dividing the polynomials p and q in x
<code>GCD[poly₁, poly₂, ...]</code>	find the greatest common divisor of the polynomials $poly_i$
<code>Resultant[poly₁, poly₂, x]</code>	find the resultant of two polynomials
<code>Factor[poly]</code>	factor a polynomial
<code>FactorTerms[poly]</code>	factor out the gcd of all terms
<code>FactorSquareFree[expr]</code>	write a polynomial as a product of powers of square-free factors
<code>FactorList[poly]</code> , <code>FactorTermsList[expr]</code> , <code>FactorSquareFreeList[poly]</code>	give results as lists of factors
<code>Cyclotomic[n, x]</code>	give the cyclotomic polynomial of order n in x
<code>Decompose[poly, x]</code>	decompose $poly$, if possible, into a composition of a list of simpler polynomials

Algebraic operations on polynomials.

Given two polynomials $p(x)$ and $q(x)$, one can always uniquely write $\frac{p(x)}{q(x)} = a(x) + \frac{b(x)}{q(x)}$, where the degree of $b(x)$ is less than the degree of $q(x)$.

`PolynomialQuotient` gives the quotient $a(x)$, and `PolynomialRemainder` gives the remainder $b(x)$.

This gives the remainder from dividing x^2 by $1 + x$.

```
In[1]:= PolynomialRemainder[x^2, x+1, x]
Out[1]= 1
```

Here is the quotient of x^2 and $x + 1$, with the remainder dropped.

```
In[2]:= PolynomialQuotient[x^2, x+1, x]
Out[2]= -1 + x
```

This gives back the original expression.

```
In[3]:= Factor[ % + %% / (x + 1) ]
Out[3]=  $\frac{x^2}{1 + x}$ 
```

`GCD[poly1, poly2, ...]` finds the largest polynomial that divides all the $poly_i$ exactly. Notice that the function `GCD` used for polynomials is the same one that we discussed for numbers in Section 3.2.4.

`GCD` gives the greatest common divisor of the two polynomials, and writes the result in expanded form.

```
In[4]:= GCD[ (1-x)^2 (1+x) (2+x), (1-x) (2+x) (3+x) ]
Out[4]= -2 + x + x^2
```

`Factor` writes the result in a form that makes its origin clear.

```
In[5]:= Factor[%]
Out[5]= (-1 + x) (2 + x)
```

The function `Resultant[poly1, poly2, x]` is needed in various algebraic algorithms. The resultant of two polynomials a and b , both with leading coefficient one, is given by the product of all the differences $a_i - b_j$ between the roots of the polynomials. It turns out that for any pair of polynomials, the resultant is also a polynomial. The resultant of two polynomials is zero if and only if the two polynomials have a common root.

Here is the resultant with respect to y of two polynomials in x and y . The original polynomials have a common root in y only for values of x at which the resultant vanishes.

```
In[6]:= Resultant[(x-y)^2-2, y^2-3, y]
Out[6]= 1 - 10 x^2 + x^4
```

The functions `Factor`, `FactorTerms` and `FactorSquareFree` perform various degrees of factoring on polynomials. `Factor` does full factoring over the integers. `FactorTerms` extracts the GCD or “content” of all the terms. `FactorSquareFree` pulls out any factors that appear squared, and then writes the polynomial as a product of powers of square-free factors.

Here is a polynomial, in expanded form.

```
In[7]:= t = Expand[ 2 (1 + x)^2 (2 + x) (3 + x) ]
```

```
Out[7]= 12 + 34 x + 34 x^2 + 14 x^3 + 2 x^4
```

FactorTerms just pulls out the factor of 2 that is common to all the terms.

```
In[8]:= FactorTerms[t]
```

```
Out[8]= 2 (6 + 17 x + 17 x^2 + 7 x^3 + x^4)
```

FactorSquareFree factors out the term $(1 + x)^2$, but leaves the rest unfactored.

```
In[9]:= FactorSquareFree[t]
```

```
Out[9]= 2 (1 + x)^2 (6 + 5 x + x^2)
```

Factor does full factoring, recovering the original form.

```
In[10]:= Factor[t]
```

```
Out[10]= 2 (1 + x)^2 (2 + x) (3 + x)
```

Particularly when you write programs that work with polynomials, you will often find it convenient to pick out pieces of polynomials in a standard form. The function **FactorList** gives a list of all the factors of a polynomial, together with their exponents. The first element of the list is always the overall numerical factor for the polynomial.

The form that **FactorList** returns is the analog for polynomials of the form produced by **FactorInteger** for integers.

Here is a list of the factors of the polynomial in the previous set of examples. Each element of the list gives the factor, together with its exponent.

```
In[11]:= FactorList[t]
```

```
Out[11]= {{2, 1}, {2 + x, 1}, {3 + x, 1}, {1 + x, 2}}
```

The function **Cyclotomic** $[n, x]$ gives the cyclotomic polynomial of order n in x . Cyclotomic polynomials arise as “elementary polynomials” in various algebraic algorithms. The cyclotomic polynomials are defined by $C_n(x) = \prod_k (x - e^{2\pi i k/n})$, where k runs over all positive integers less than n that are relatively prime to n .

This is the cyclotomic polynomial $C_6(x)$.

```
In[12]:= Cyclotomic[6, x]
```

```
Out[12]= 1 - x + x^2
```

$C_6(x)$ appears in the factors of $x^6 - 1$.

```
In[13]:= Factor[x^6 - 1]
```

```
Out[13]= (-1 + x) (1 + x) (1 - x + x^2) (1 + x + x^2)
```

Factorization is one important way of breaking down polynomials into simpler parts. Another, quite different, way is *decomposition*. When one factors a polyno-

mial $P(x)$, one writes it as a product $p_1(x)p_2(x)\dots$ of polynomials $p_i(x)$. Decomposing a polynomial $Q(x)$ consists in writing it as a *composition* of polynomials of the form $q_1(q_2(\dots(x)\dots))$.

Here is a simple example of `Decompose`.

The original polynomial $x^4 + x^2 + 1$ can be written as the polynomial $\bar{x}^2 + \bar{x} + 1$, where $\bar{x}$ is the polynomial x^2 .

```
In[14]:= Decompose[x^4 + x^2 + 1, x]
```

```
Out[14]= {1 + x + x^2, x^2}
```

Here are two polynomial functions.

```
In[15]:= ( q1[x_] = 1 - 2x + x^4 ;
           q2[x_] = 5x + x^3 ; )
```

This gives the composition of the two functions.

```
In[16]:= Expand[ q1[ q2[ x ] ] ]
```

```
Out[16]= 1 - 10 x - 2 x^3 + 625 x^4 + 500 x^6 + 150 x^8 +
          20 x^10 + x^12
```

`Decompose` recovers the original functions.

```
In[17]:= Decompose[%, x]
```

```
Out[17]= {1 - 2 x + x^4, x (5 + x^2)}
```

`Decompose[poly, x]` is set up to give a list of polynomials in x , which, if composed, reproduce the original polynomial. The original polynomial can contain variables other than x , but the sequence of polynomials that `Decompose` produces are all intended to be considered as functions of x .

Unlike factoring, the decomposition of polynomials is not completely unique. For example, the two sets of polynomials q_i and q'_i , related by $q'_1(x) = q_1(x - a)$ and $q'_2(x) = q_2(x) + a$ give the same result on composition, so that $q_1(q_2(x)) = q'_1(q'_2(x))$. *Mathematica* follows the convention of absorbing any constant terms into the first polynomial in the list produced by `Decompose`.